

Second Brain The Executable Infographic Factory

Turning a Custom GPT into an interface that runs

Dr. Joseph Oduor Odongo

July 2026

Second Brain — Executable Infographic Factory

The one sentence. Give the system a Custom GPT's name, description, and URL; get back an infographic that **runs**: the GPT wired in the centre, morphic simulation panels around it, everything downloadable.

This is the map of content. Each note below is a single idea that stands on its own. Follow the links.

The doctrine

- **The Design Law** — never embed the simulation in the image
- **The Local Estimator Doctrine** — why every build ships a `mockRun`
- **The Retarget Invariant** — what may change, what may never change

The architecture

- **Core, Ring, and Bus** — the three-part anatomy
- **The Action Schema Bridge** — one contract, two front-ends
- **The Morphic Panel Contract** — the interface every panel implements
- **The Archetype Taxonomy** — how a GPT description becomes a panel set

The practice

- **The Six-Stage Pipeline** — what Claude Code actually executes
- **The Validation Gate** — what must pass before a build ships
- **Deployment Paths** — local, GitHub Pages, HF Space

The evidence

- **Case Study — RNAi Precision Crop Designer** — a static poster, executed
- **Open Threads** — what is not solved yet

Provenance

Built July 2026. Three reference builds exist: a regulatory risk screener, a football odds model, and the RNAi crop designer. All three share identical machinery — that is the claim this vault exists to defend.

The Design Law

Never embed the simulation, model, or GPT inside the image. Make the infographic a visual interface that links to interactive experiences.

A PNG or JPEG is static. That is not a defect to be engineered around — it is the correct role. The image's job is to be the **entry point**, not the container.

Why this matters more than it sounds

The tempting failure is to try to cram interactivity into the artifact: animated GIFs, embedded video of a simulation, a

screenshot of a chart with a caption saying “interactive version available.” Each of these degrades the thing it imitates and teaches the reader that the interactivity is decorative.

The inversion: build the **live app first** (`app.html`), and let the static image be an **export** of it. The poster is downstream of the interface, not upstream. A QR code or a link on the poster routes back to the live thing.

The test

Point at any element of the infographic and ask: **if I click this, does something happen?** If the honest answer is “no, that’s a picture of a button,” the design law has been violated — and the reader will find out, which is worse than never having promised.

See **Case Study — RNAi Precision Crop Designer** for what this looks like when a static poster is put through the factory.

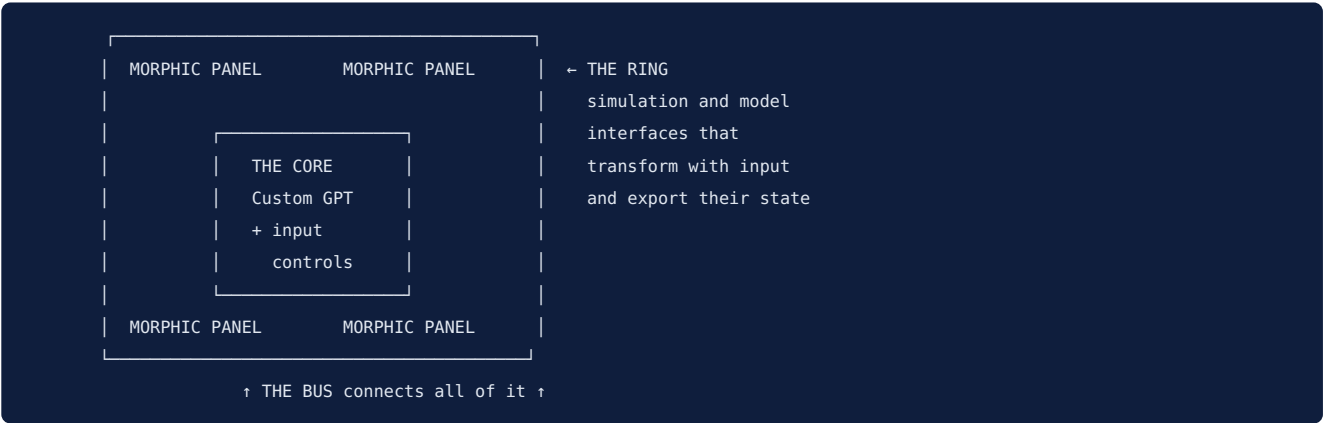
Consequences

- The deliverable is `app.html`, not a raster file.
- The poster PNG is generated **from** the app.
- Every affordance in the layout is wired or removed. No exceptions.

Related: **Core, Ring, and Bus, The Validation Gate**

Core, Ring, and Bus

Every build has the same anatomy. Three parts, one mechanism connecting them.



The core

The Custom GPT’s presence on the page. It renders the input contract as controls and holds two routes to the GPT: a live call to its Action backend, and a deep-link to the full conversational GPT. See **The Action Schema Bridge**.

The ring

Three to five **morphic panels** — simulations, charts, models. Not decoration. Each one subscribes to the bus and re-renders on every change. Each exports its current state. See **The Morphic Panel Contract**.

The bus

The whole system in twelve lines. One shared state object, a subscriber list, and a `set` that notifies everyone.

```
const Bus = (() => {
  const state = {}; const subs = [];
  return {
    set(patch){ Object.assign(state, patch); subs.forEach(fn => fn(state)); },
    get(){ return {...state}; },
    subscribe(fn){ subs.push(fn); fn(state); }
  };
})();
```

This is why “user inserts inputs, which is reflected in the interfaces” is true rather than aspirational. There is exactly one source of truth. A panel cannot drift out of sync with the core because it has no state of its own to drift with.

The rule that keeps it honest

Every panel must read at least one bus key. A panel that renders the same thing regardless of input is a picture, and pictures belong in **The Design Law**’s list of failures. The validation gate rejects them.

Related: **The Morphic Panel Contract**, **The Retarget Invariant**

The Action Schema Bridge

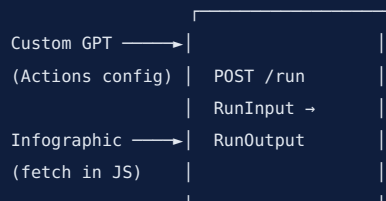
The constraint nobody mentions

OpenAI Custom GPTs serve `X-Frame-Options: DENY`. **You cannot iframe a Custom GPT.** Any design that assumes “embed the GPT in the middle of the page” is dead on arrival, and discovering this after building the layout is expensive.

The move

Don’t embed the GPT. Embed **the contract the GPT uses**.

A Custom GPT with an Action calls a backend — a plain HTTP endpoint described by an OpenAPI schema. That schema is a public, reusable artifact. So:



One schema. Two front-ends. The GPT and the infographic call the same endpoint with the same body and get the same shape back. The infographic is not **pretending** to be connected to the GPT — it is connected to the GPT’s brain.

The two modes

Live mode (`action_base_url` is set). The core `fetch`es `/run` and renders `RunOutput` inline. The panels visualize the **real** model output.

Deep-link mode (no backend yet). The core runs the local estimator (see **The Local Estimator Doctrine**), copies a formatted prompt to the clipboard, and opens the real GPT in a new tab. The page is fully interactive before any backend exists — which means you can ship the infographic on day one and wire the backend on day thirty.

The schema is the deliverable

Every build emits `action.schema.json`. Two consumers: 1. Paste it into the Custom GPT’s **Actions** config. 2. It is the request/response contract the infographic’s `fetch` obeys.

Keep the property `description` fields precise — units, ranges, enums. The GPT reads them to decide **when and how** to call the action. Vague descriptions produce a GPT that calls the action wrongly or not at all.

CORS

The infographic is served from a different origin than the backend. The backend **must** send permissive CORS headers on `/run` or the browser will refuse the call. This is the single most common reason a live-mode build silently falls back to the estimator.

Related: [Core, Ring, and Bus](#), [Deployment Paths](#)

The Morphic Panel Contract

Every panel — a gauge, a dose curve, a heatmap, a sequence track — implements the same four-member interface. This is what makes the download layer and the bus wiring **identical across every build**.

```
{
  id:      "verdict-gauge",      // unique; also the export filename
  accepts: ["result", "gc"],    // bus keys it reads
  mount(el, cfg) { ... },      // build DOM once
  render(state) { ... },       // called on EVERY bus change
  export() { ... }             // → {png?, csv?, json?, html?}
}
```

Why a contract and not just “write some charts”

Without it, every build reinvents its gauge, its download button, its wiring — and each reinvention is a fresh opportunity for drift. With it:

- **The bus wiring never changes.** `PANELS.forEach(p => Bus.subscribe(p.render))`.
- **The download layer never changes.** One `downloadPanel(p)` serves all of them.
- **Panels compound.** A panel written for one build is available to the next.

That compounding is the whole economic argument for the factory. Build three, and the fourth is nearly free.

The library

Implemented and validated: `param-sliders`, `verdict-gauge`, `factor-bars`, `threshold-slider-curve`, `distribution-curve`, `dose-curve`, `sensitivity-heat`, `outcome-table`, `before-after`, `stage-stepper`, `stage-wheel`, `sequence-track`, `decay-curve`.

Described but not yet written: `map-canvas`, `route-list`, `diff-view`, `token-meter`, `concept-graph`, `progress-meter`, `timeline-canvas`.

The discipline

Reuse before you write. Before adding a panel, check the library. A reused panel is already validated; a new one is a new liability. When a genuinely new view is needed, write it **to the contract**, add it to the library, and log it in the ledger — so the next build inherits it.

Downloadable by construction

“The transform interfaces are downloadable” is not a feature bolted on at the end. It is the `export()` member of the contract. A panel that cannot export its state has not implemented the interface.

Related: [Core, Ring, and Bus](#), [The Retarget Invariant](#)

The Archetype Taxonomy

The analyzer’s job: read a GPT’s description and decide **what kind of thing it is**, because that determines which panels can possibly morph.

Archetype	Signals in the description	Native input	Native output	Default panels
<code>risk-screener</code>	screen, assess, dossier, flag, comply	a case	verdict + score	verdict-gauge, factor-bars, threshold-slider-curve, case-table
<code>quant-model</code>	predict, odds, probability, forecast	numeric params	distribution	distribution-curve, param-sliders, sensitivity-heat, outcome-table
<code>dose-response</code>	dose, EC50, exposure, toxicity	dose vector	response curve	dose-curve, param-sliders, threshold-marker
<code>pipeline-designer</code>	pipeline, stages, workflow, design → validate	construct params	staged result	stage-wheel, sequence-track, dose-curve, decay-curve
<code>geo-ops</code>	border, route, region, map	places	map + route	map-canvas, route-list, param-sliders
<code>text-transform</code>	rewrite, summarize, extract, classify	text	text	before-after, diff-view, token-meter
<code>explainer</code>	teach, explain, tutor	topic + level	staged explanation	stage-stepper, concept-graph, progress-meter
<code>generic</code>	(nothing matched)	—	—	param-sliders, series-chart, outcome-table

The judgement call

The taxonomy is a starting point, not a cage. A GPT can straddle two archetypes — the RNAi designer is a `pipeline-designer` with a `dose-response` panel embedded in it. When that happens, take the pipeline’s **shape** and borrow the other’s **panels**. The archetype decides the **layout**; the input/output contract decides the **panels**.

When nothing fits

Add an archetype. `pipeline-designer` did not exist until a staged workflow needed it; it now exists, with a wheel component, for every staged GPT that follows. The taxonomy is supposed to grow. What it must never do is get **skipped** — an unclassified build has no principled basis for its panel selection, and it shows.

Related: [The Morphic Panel Contract](#), [The Six-Stage Pipeline](#)

The Local Estimator Doctrine

Every build ships a `mockRun(inputs)` — a small, deterministic function that computes the domain outcome locally, in the browser, with no backend.

Why it exists

Without it, an infographic with no backend is dead: the sliders move and nothing happens. The estimator means **the page is fully interactive on day one**, before any endpoint is deployed, before the GPT’s Action is configured. You can hand the file to someone and it works offline.

It is also the **only** place in a build where real domain thinking happens. The machinery is generic; the estimator is not. Writing it forces you to state, in executable form, what you actually believe about the domain.

The three requirements

Deterministic. Same inputs, same output, every time. No `Math.random()` in the output path. Where randomness is needed (generating a sequence), seed a PRNG from the inputs so the construct is reproducible.

Transparent. A reader should be able to open the source and see the model. Weighted logistic, Poisson grid, Hill equation — legible, small, arguable.

Labeled. This is the non-negotiable one. The UI must say **local estimate** and must point at the GPT as the authoritative source. An estimator presented as the GPT’s answer is a lie with a chart on it.

The line that must not be crossed

The estimator makes the interface work. It does not make the interface right.

When the real backend lands, `mockRun` becomes a fallback for network failure — not a competitor to the model. If your estimator is **good enough to ship as the answer**, then it isn't an estimator, it's the model, and it belongs behind `/run` where the GPT can reach it too.

Validate it outside the browser

Run the estimator in `node` across edge cases before shipping. Check monotonicity: does tightening a constraint move the outcome in the direction it should? A model that is subtly non-monotonic will produce a panel that looks fine and teaches the reader something false.

Related: [The Action Schema Bridge](#), [The Validation Gate](#)

The Retarget Invariant

Change `CFG`. Change the theme. Change the panel selection. Never change the Bus, the panel contract, or the download layer.

This one rule is the difference between a factory and a folder of one-off pages.

What varies

Varies per build	Never varies
<code>CFG.gpt</code> — name, url, action_base_url	the Bus
<code>CFG.inputs</code> — the input contract	the panel contract (<code>mount/render/export</code>)
<code>CFG.mockRun</code> — the domain model	the download layer
<code>CFG.promptTemplate</code>	the core's live/deep-link dual mode
the theme (palette, type)	the grid → single-column collapse
which panels from the library	how panels are wired

The proof

Three builds exist. They look nothing alike — a forest-green regulatory instrument, a navy trading terminal, a white-and-blue research poster. Diff their machinery and it is byte-identical. Diff their `CFG` and they share almost nothing.

That is the invariant holding.

Why the theme must change

Two builds that look alike defeat the purpose. An infographic's job is to be **this** subject's interface, and a subject's world — its instruments, its vernacular, its materials — is where a distinctive palette comes from. Shipping the same template skin twice is the visual equivalent of skipping the analyzer.

Specifically: avoid the default AI-design tells — cream background, high-contrast serif, terracotta accent. They appear regardless of subject, which is exactly the problem.

Related: [Core, Ring, and Bus](#), [The Morphic Panel Contract](#)

The Six-Stage Pipeline

What Claude Code executes when pointed at an intake note. Each stage consumes the previous stage's output. The order

is not negotiable.

Stage	Reads	Emits
0 • Intake	the user's 3 fields	—
1 • Analyzer	intake	<code>build.spec.json</code> — archetype, inputs, outputs, panel picks
2 • Schema forge	<code>build.spec</code>	<code>action.schema.json</code> — OpenAPI 3.1
3 • Embedder	<code>build.spec</code> + schema	the core config: bus, live/deep-link modes
4 • Morphic panels	<code>build.spec</code>	selected panels, wired to the bus
5 • Build	everything	<code>app.html</code> + <code>README.md</code>
6 • Validate	<code>app.html</code>	pass/fail report + ledger entry

The user supplies three fields

```
gpt_name:      ""
gpt_description: ""
gpt_url:       "https://chatgpt.com/g/..."
action_base_url: "" # optional
```

That is the entire input surface. No schema, no HTML, no JS. Everything else is inferred — which is the point: the analyzer's inference **is** the product.

Where the thinking actually goes

Stages 2-5 are largely mechanical once Stage 1 has decided the archetype and the input contract. The intellectual work is concentrated in: - **Stage 1**, choosing the archetype and the input contract, and - the `mockRun` written during Stage 3 (see **The Local Estimator Doctrine**).

Get those two right and the rest assembles.

Write it down

Stage 6 appends to the **panel ledger**: which panels this build used, which new ones it added to the library. Without the ledger, the library does not compound — the next build reinvents rather than inherits.

Related: **The Archetype Taxonomy**, **The Validation Gate**

Case Study — RNAi Precision Crop Designer

A static poster went into the factory. An executable infographic came out. This note records what changed, because the delta **is** the argument.

The delta

In the poster	In the build
7-stage wheel is an illustration	seven clickable, keyboard-focusable stages; the methodology list drives the same wheel
four Launch buttons	four simulators already running, re-deriving on every input
96.3% printed as text	live silencing prediction from the current construct
QR codes are drawn squares	real scannable QR to the paper, the GPT, and the live app
Run Full Workflow Simulation	animates the wheel through all seven stages, then runs the model
one static image	one shared bus; six inputs feed everything

The input contract

GC content · construct length · applied dose · BLAST stringency · field temperature · host crop. Six inputs, one bus, four panels plus the wheel plus the predictive-model readout all subscribing.

The four morphic panels

- 1. **dsRNA design** — a PRNG seeded from GC + length, so the same inputs always produce the **same** construct. Reproducible, not decorative. A sliding-window GC track plots beneath the sequence.
- 2. **Off-target** — BLAST hit count against stringency, with the current stringency marked. Exports the whole curve.
- 3. **Efficacy** — dose-response gated by stability, specificity, and a temperature factor peaking near 26 °C. EC50 marked.
- 4. **Environmental impact** — soil persistence decay over 21 days, banded LOW / MODERATE / HIGH by non-target exposure.

Validation (run in node, before shipping)

case	silencing	off-target hits	specificity	stability	persistence
defaults	49.0%	6	50%	93%	5.2 d
stringency 45%	41.9%	18	25%	93%	5.2 d
stringency 95%	63.3%	0	100%	93%	5.2 d
GC 70 / 800 bp	19.1%	12	33%	21%	5.2 d
dose 120, tight	87.0%	0	100%	100%	4.5 d
field 10 °C	41.8%	1	86%	100%	12.4 d

Monotonic in every direction it should be: tightening stringency raises efficacy and collapses hits; a bad GC on an over-long construct craters stability; a cold field triples soil persistence.

The honest caveat

The efficacy model here is a **placeholder estimator**, labeled as such in the UI. The real model — the one behind the poster’s 96.3% — belongs behind `/run`, where both the GPT and the infographic can reach it. See **The Local Estimator Doctrine** and **The Action Schema Bridge**.

What it contributed back

A new archetype (`pipeline-designer`) and three new library panels (`stage-wheel` , `sequence-track` , `decay-curve`). The next staged-workflow GPT gets them free.

Deployment Paths

`app.html` is a single self-contained file with no build step. That constraint buys a lot of deployment freedom.

The infographic

Path	How	Good for
Local	double-click the file	review, offline demo, air-gapped settings
GitHub Pages	commit to <code>/docs</code> or <code>gh-pages</code> , enable Pages	free, versioned, public
HF Static Space	create a Static Space, push <code>app.html</code> as <code>index.html</code>	free, sits next to the backend
Own domain	drop the file anywhere that serves static assets	branding, permanent QR targets

The backend (only for live mode)

A FastAPI app exposing `POST /run` , validating against `RunInput` , returning `RunOutput` . Deploy on a HuggingFace Docker Space or any host.

Two things that will bite you: 1. **CORS.** The infographic is a different origin. Without permissive CORS on `/run`, the browser blocks the call and the build silently falls back to the estimator. Set it explicitly. 2. **The same URL goes in two places** — the GPT's Actions config **and** `CFG.gpt.action_base_url`. Update one, forget the other, and the GPT and the infographic quietly disagree about reality.

The QR codes

Generate them at build time (`segno`, pure Python, inline SVG data-URI — no CDN, no network at render time). Point them at the **live app**, not at the poster. A QR code that leads back to a picture is a joke the reader will get.

Versioning

The infographic is a document **and** a program. Tag releases. A reader who scans a QR in a printed poster next year should land on something that still runs — which means the deployed `app.html` needs a stable URL and the backend needs a stable contract. Breaking `RunInput` breaks every poster ever printed.

Related: [The Action Schema Bridge](#), [The Design Law](#)

The Validation Gate

Nothing ships with a failure in the **must** set.

Must pass

1. **Opens offline.** Double-click `app.html`; it renders; no console errors.
2. **Bus propagation.** Changing any core input visibly re-renders at least two panels.
3. **Morph, not decorate.** Every panel reads ≥ 1 bus key. A static panel is a picture — cut it. (See [The Design Law](#).)
4. **Core wired correctly.** Live mode calls `/run` and renders `RunOutput`; deep-link mode opens the GPT and copies the prompt.
5. **Downloads work.** Every panel's export produces a non-empty file.
6. **Schema round-trips.** `action.schema.json` is valid OpenAPI 3.1 and its `RunInput` matches the core's request body exactly.
7. **Model is monotonic.** Run `mockRun` in node across edge cases. Constraints must move outcomes in the direction they should.
8. **Mobile.** Single column below $\sim 860\text{px}$, core first, no horizontal scroll.

Should pass

9. Reduced motion honored; keyboard focus visible; every control labeled.
10. Theme is domain-specific, not the generic default. (See [The Retarget Invariant](#).)
11. The exported poster contains a route back to the live app.

The check that catches the most

#3. It is easy to add a panel that looks like data and is in fact a constant. It renders, it exports, it passes every other test — and it teaches the reader that the interactivity is theatre. Grep every panel for a bus key. If it doesn't read one, it doesn't ship.

Then log it

Append to the panel ledger: which panels were used, which were invented. This is how the library compounds instead of forking.

Related: [The Six-Stage Pipeline](#), [The Local Estimator Doctrine](#)

Open Threads

Honest accounting of what is not solved.

Panels described but not written

`map-canvas`, `route-list` — the biggest hole. A `geo-ops` build (border management, route planning) is blocked on these. `diff-view`, `token-meter`, `concept-graph`, `progress-meter`, `timeline-canvas` are also outstanding.

Poster export is browser-limited

Rasterizing the full app container to PNG via `foreignObject` is fragile across browsers. Current fallback: tell the user to screenshot. A real fix likely means drawing the poster layout directly to canvas rather than serializing the DOM — which means the poster becomes its own render target, not a snapshot. Arguably that is the more honest architecture anyway.

Prompt prefill into ChatGPT is unreliable

Deep-link mode copies a formatted prompt to the clipboard and opens the GPT. There is no dependable URL-prefill for Custom GPTs. The clipboard hop is a papercut and the user may not notice the copy happened. Needs a visible confirmation.

Multi-GPT builds

The original brief allowed **multiple** GPTs as input. Every build so far has one. A two-GPT infographic — one designing, one screening — would need a core that routes inputs to the right backend and a bus that namespaces results. Not hard; not yet done.

The estimator/model boundary needs enforcement

Nothing currently **stops** someone shipping a `mockRun` presented as the real model. The doctrine (**The Local Estimator Doctrine**) is a norm, not a mechanism. A lint rule — “if `action_base_url` is null, the answer element must contain the string ‘local estimate’” — would make it structural.

Accessibility beyond the floor

Keyboard focus and reduced motion are handled. Screen-reader narration of a canvas panel that just morphed is not. A live region announcing “silencing efficiency now 63 percent” would matter for the educational use case this is aimed at.